Chunking in RAG :->

Why?

-> When we embed text, embedding models have token limits. Feeding entire documents isn't possible, hence we split documents into chunks, embed them and use vector search to retrieve relevant pieces.

If chunking:

-> Too small -> context ↓ irrelevant answers.

-> Too large -> context ↑ noise ↑ cost ↑ (retrieval)

-> Poor boundaries -> semantic meaning gets cut mid sentences.

Types :->

a) Fixed-size chunking

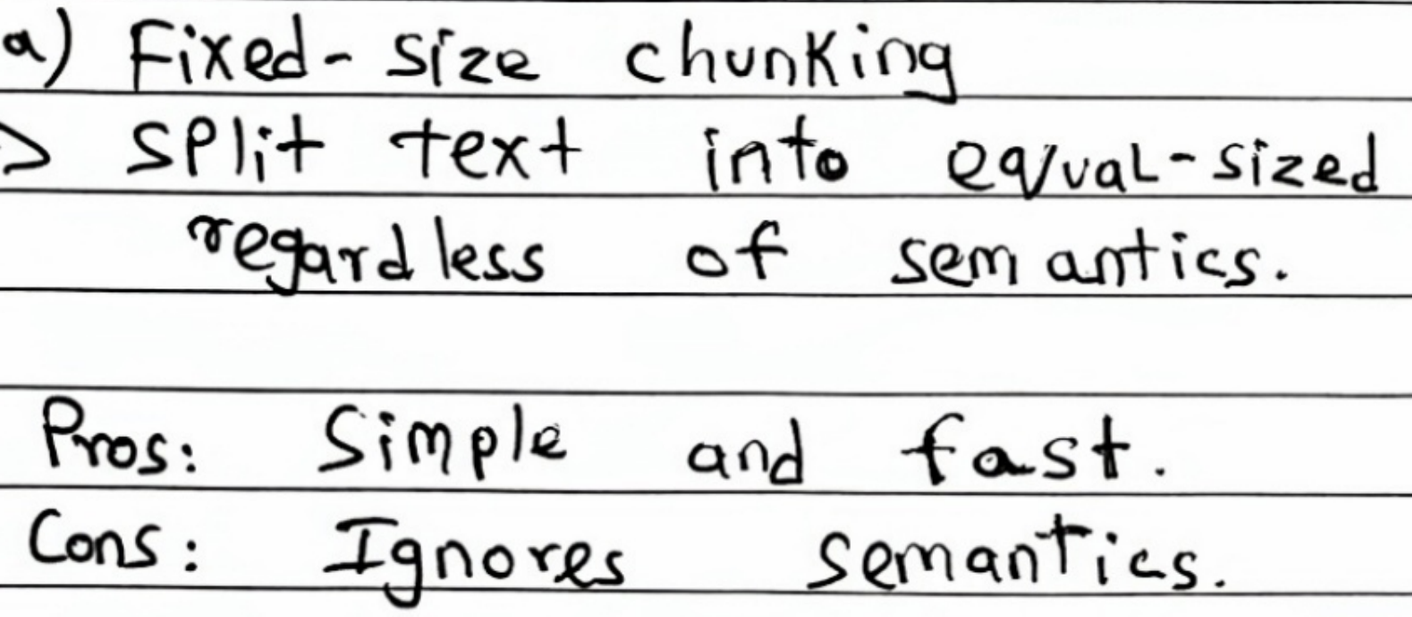
-> Split text into equal-sized segments, regardless of semantics.

Pros: Simple and fast.

Cons: Ignores semantics.

Use: (a) Structured / uniform text.

(b) Speed > Precision.



M T W T F S S

Page No.: YOUVA

Date:

b) Recursive Character splitting.

→ Split text hierarchally, first by paragraphs, then by sentences, then by characters until chunks fit within size limit.

Pros: → Preserves natural boundaries
→ Widely used

Cons: Performance overhead for large Docs:-s

Use: → Mixed Docs
→ General Purpose RAG- Pipelines.

c) Semantic ChunKing.

→ Use embeddings to segment text by the topic similarity.

→ Sentences that are semantically close are grouped.

Pros: → Preserves semantic meaning
→ Improves retrieval precision

Cons: → Computationally expensive
→ Harder to tune

Use: → Knowledge Bases, FAQs, Research Papers.

— X —

M T W T F S S

Page No.: YOUVA

Date:

d) Language Based Chunking :->

-> Uses NLP Tools to create splits that makes sense to a human reader.

Pros: -> Human-friendly, readable.

Cons: -> Cannot adopt to semantic shifts, Also exceeds embedding model limits.

Use :-> legal, conversational documents.

e) Context-Aware chunking.

-> Uses Document's Internal structure to Decide exactly where to make cut, ensuring that every chunk contains Complete, logical piece of Information.

Pros: -> Domain specific
-> Highest retrieval relevance

Cons: -> More engineering effort

Use: -> Domain-specific RAG
-> when meta data is available.

—— X ——

#RAG Retrieval :->

M T W T F S S

Page No.:

Date:

YOUVA

Problems for Production :

-> Accuracy
-> latency
-> Scaling & cost
-> Access over Documents.

1.) BM 25 - Lexical Search

-> Build an Inverted Index :
list [word -> list of docs/chunks containing that word]
-> Scores using -> length normalisation
-> Term freqvency.

Pros: -> extremely fast & strong Baseline
-> search engines's optimized.

2.) Embeddings & Semantic Search

-> Convert text -> vector (embedding)
similar meaning vectors are near to each other.

Types: a) Static embeddings (word2Vec)
-> one fixed vector per word
Pros: -> fast & cheap
Cons: -> No context

b) Sentence Transformers (dense retrieval)
-> encodes sentence / chunk
-> Better for retrieval when keywords differ.

c) Matryoshka embeddings :->
->Reduce Dimension while Keeping Quality.

Pros: less storage + faster retrieval with less quality loss

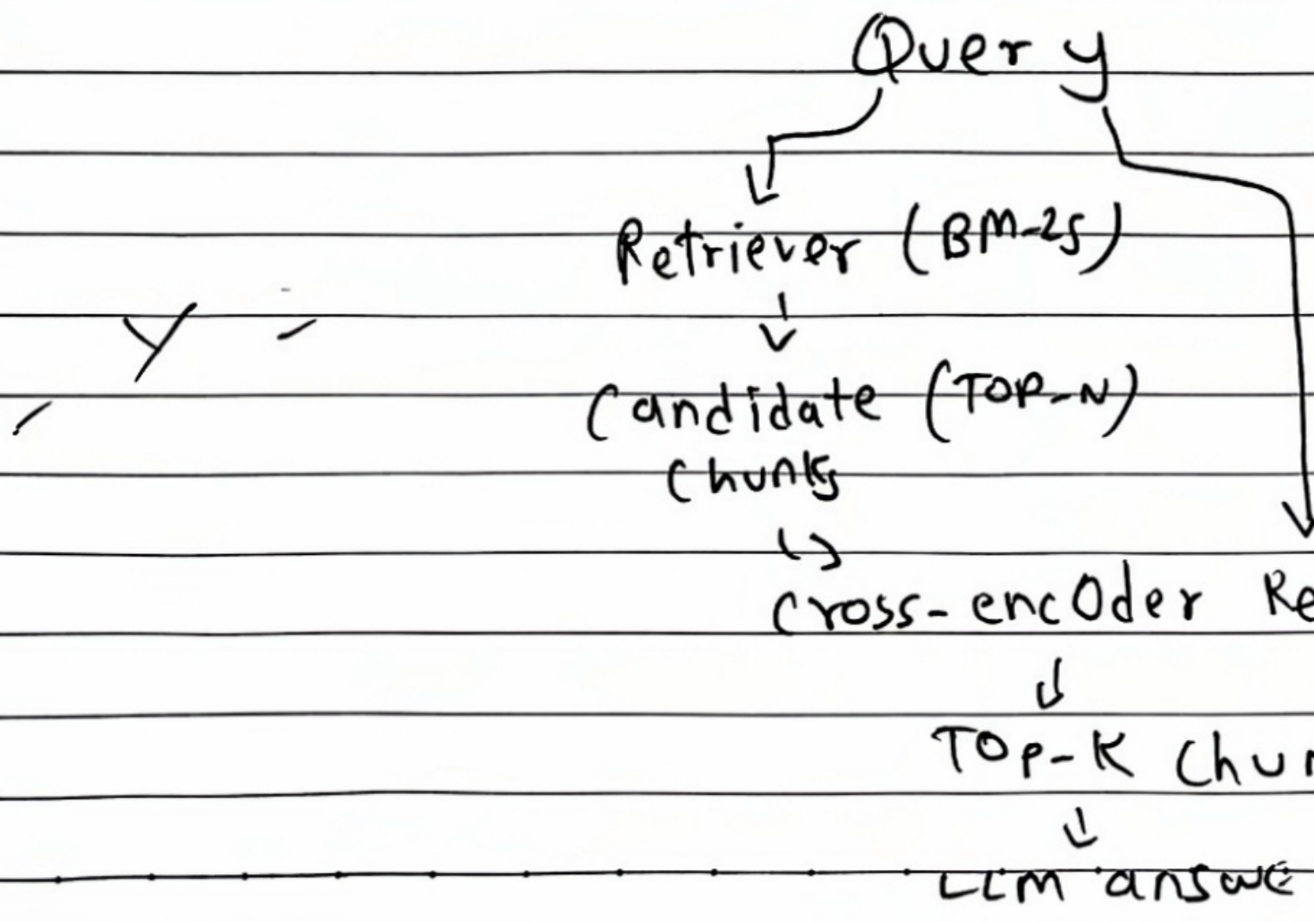
Cons: Misses the exact fact if chunking/ metadata is weak.

3) Cross-Encoders (Re-Rankers)

Purpose: Improve Ranking after the initial retrieval.

Steps: a) Retrievar fetches candidate chunks (BM-25)
b) Cross-encoder reads Both (Query, chunk)
c) Produces relevance scores
d) Re-Ranks -> Keep top-k

Higher accuracy But latency also.



4) Hybri

RRF -> C

* Metadata
->Pre- Filt

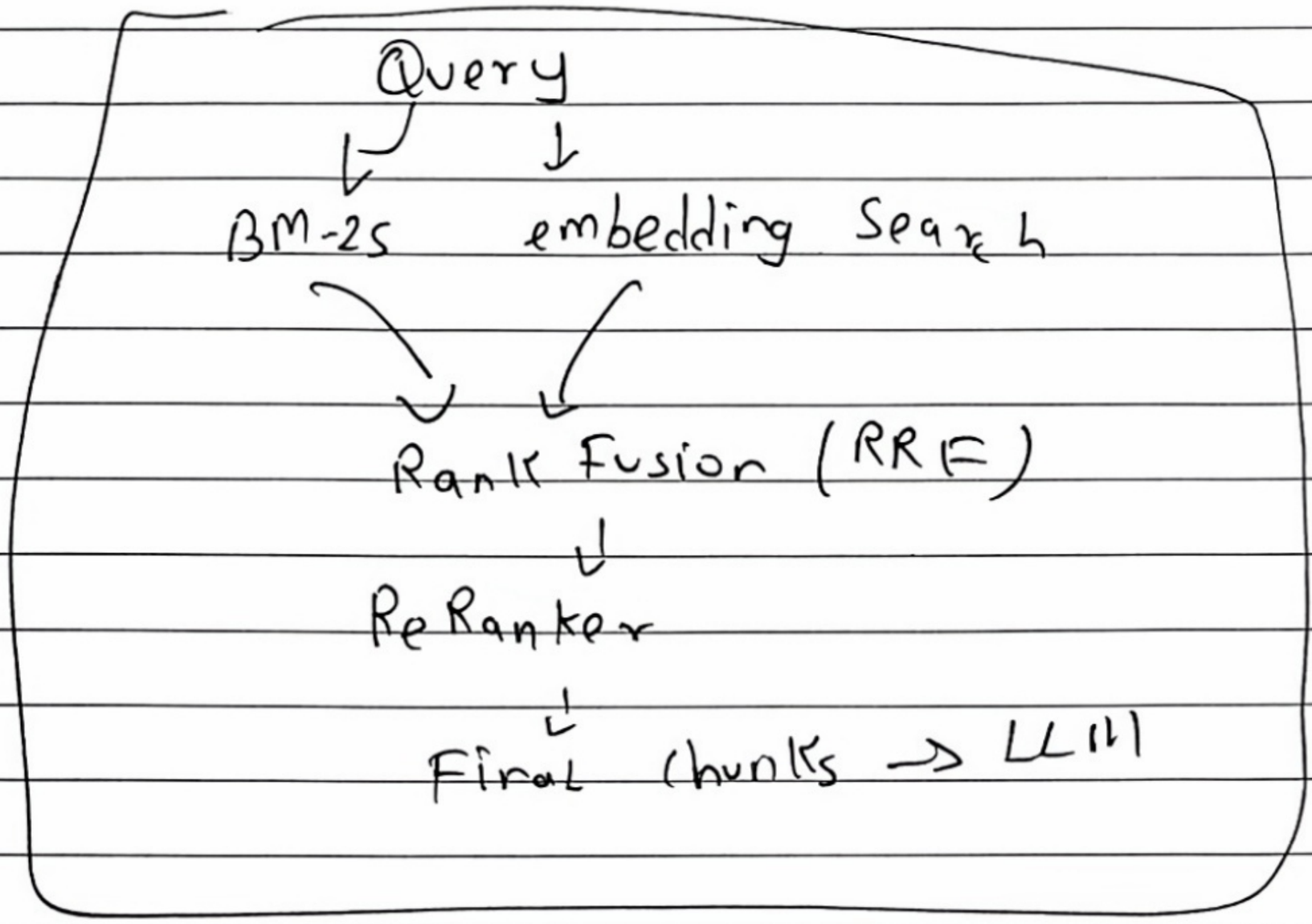
M T W T F S S

Page No.:

Date:

YOUVA

4) Hybrid Search (Recommended)



RRF -> Combine Ranks from multiple retrievers

* Metadata- Filtering
->Pre- Filter to reduce search space

M T W T F S S

Page No.: YOUVA

Date:

Retrieval evaluation (RAG)

Quality → Retrieval (which chunks got fetched)

→ Generation (LLM use)

-> for evaluation we need
(Query, expected doc/chunk)

Query -> Retriever → Ranked results → Compare
→ expected results →

↓

Score Per Query

↓

Aggregate metrics.

Recall @ K
↳ checks coverage

MRR
↳ checks the Ranking Quality.

Recall @ K
-> Queries with expected ones in Top K / N

{ If expected in Top K -> 1
If not -> 0 }

ex

R
-> I

M

ex:

MR

t n

ex: K = 3

Query	Top-3	expected	Recall @ 3
Q_1	[A, B, C]	A	1
Q_2	[D, E, F]	F	1
Q_3	[G, H, I]	Z	0

∴ Recall @ 3 = 2/3 = 0.667

MRR (Mean Reciprocal Rank)
-> It rewards putting the correct Item near the top.

$$MRR = \frac{1}{N} \sum_{i=1}^{N} \frac{1}{rank_i}$$

ex: Ranks of expected [1, 2, 3, 5, not found]

$$MRR = \frac{1}{N} \left(1 + \frac{1}{2} + \frac{1}{3} + \frac{1}{4} + 0\right] = \frac{25}{12 \times 5}.$$

$$= 0.406$$

** Recall @K low → focus on chunking / embedding / Query / filters

MRR low → focus on soring / Re-Ranking.

— x —